PROGRAMACION IV - General
1: Introducción a FastApi+...
2: SQLModel + React + Tail...
3- Relaciones y Estado
4- TYPESCRIPT
5- To String - Equal y hashc...
6- Colecciones
8- Reflection y Genéricos
7- Lombock y DTO
Programación Funcional
JPA
Fundamentos Spring Boot
APIs REST Spring Boot
Evaluaciones
Entrenamiento

🛠️ Trabajo Práctico – FastApi

Marcar como hecha

?

🛠️ Trabajo Práctico – Ejercicio Integrador FastAPI 🚀

En esta actividad trabajaremos sobre una API modular desarrollada con FastAPI, aplicando todos los conceptos vistos en la unidad.

🎥 Video explicativo

- ▶️ **U1.Video 7 – Ejercicio Integrador (Estructura modular + Routers + Services)**
- 📌 **Filmina utilizada en el video.**

📂 Repositorio base

📎 Ejemplos e Integrador – Repositorio oficial

El proyecto a utilizar se encuentra dentro de:
u1_ej_8_integrador

📌 Paso 1 – Probar el sistema base

- Clonar el repositorio.
- Instalar dependencias.
- Ejecutar el servidor con Uvicorn.
- Probar los endpoints en Swagger y Rest Client.
- Comprender el flujo: **main** → **router** → **service** → **response**.

📌 Paso 2 – Agregar nueva funcionalidad

Una vez validado el funcionamiento base, deberá agregar nueva funcionalidad respetando la arquitectura modular.

- Crear un nuevo módulo (clientes, ventas, proveedores, etc.).
- Agregar filtros avanzados con parámetros de query

?

- Agregar filtros avanzados con parámetros de query.
- Incorporar nuevas reglas de negocio en services.
- Agregar validaciones adicionales.
- Implementar nuevos endpoints con response model y manejo de errores.

⚠️ La solución debe respetar:

- Separación entre router y service.
- Modelos Pydantic correctamente definidos.
- Códigos HTTP adecuados.
- Manejo de errores con HTTPException.

Entrega proyecto final FastApi

Pendiente de hacer

?

📝 Entrega – Trabajo Práctico Integrador

La entrega deberá realizarse en un único archivo **.pdf**.

📥 El PDF debe incluir:

- Descripción breve de la funcionalidad agregada.
- Capturas de pantalla de Swagger mostrando los nuevos endpoints.
- Ejemplo de request y response correctos.
- Ejemplo de manejo de error (404 o 422).
- Enlace al repositorio de GitHub con el proyecto completo.

⚠️ El repositorio debe contener:

- Estructura modular correcta.
- Código limpio y organizado.
- Archivo `requirements.txt` actualizado.

> ⚡ **Consejo:** Verifique el correcto funcionamiento en Swagger antes de entregar. Pruebe casos válidos e inválidos para validar reglas de negocio y manejo de errores.

◄Actividades 🧩 Ir a... Autoevaluación 🔍►

?